

Scholar: RAG Pipeline & Retrieval Architecture Manual

Exhaustive Code Specification, BM25 Formulation & Provenance Grounding

100% Code-Accurate Technical Reference for Scholar Research Assistants

Executive Summary & Architectural Guarantees

Scholar uses a custom local-first Retrieval-Augmented Generation (RAG) pipeline designed around three foundational guarantees:

- **100% Citation Existence by Construction:** Eliminates LLM-generated page hallucinations by constraining the model to ephemeral evidence IDs ([E1], [E2]) resolved deterministically to immutable PDF pages in the application layer.
- **High-Recall Lexical BM25 Primary Scoring:** Prevents semantic drift on exact scientific variables (d_k , p_θ), numbers (*BLEU 28.4*), dataset names (*TruthfulQA*), and architecture tokens (*FlashAttention*).
- **Dual-Path Multimodal Routing:** Seamlessly routes figure and table queries to a local Vision LLM with high-resolution 3× zoom image crops.

1 System Architecture & Code Map

Every stage of the RAG pipeline is implemented across dedicated modules in `backend/` and rendered interactively in `frontend/`:

Component	Source File	Core Functionality & Code Reference
Orchestrator	<code>backend/main.py</code>	FastAPI endpoint <code>chat()</code> (L760–938); candidate merging, dual-path routing, evidence distillation, citation normalization.
PDF Processing	<code>backend/services/pdf_service.py</code>	PyMuPDF extraction (L193–202), 3× zoom figure cropping (L243–329), word-level bounding box highlighting (L356–470).
Chunking Engine	<code>backend/services/chunking_service.py</code>	Page-preserving chunking (L48–102), section classification (L17–38), figure chunking (L105–149).
Retrieval Engine	<code>backend/services/retrieval_service.py</code>	BM25 scoring (L160–192), CamelCase tokenization (L69–84), query expansion (L87–91), heuristic re-ranking (L263–307).
Vision Service	<code>backend/services/vision_service.py</code>	Local VLM orchestration (L87–204); base64 image encoding, multimodal prompt construction, caption fallback.
LLM Engine	<code>backend/services/ollama_service.py</code>	Local Ollama client (L39–54) and structured 8-goal guided study planner with deterministic fallback (L104–355).
Presentation	<code>frontend/components/ChatBox.tsx</code>	KaTeX math rendering (L48–86), interactive citation chips (L92–100), figure thumbnail cards.

2 Document Ingestion & Page-Preserving Chunking

2.1 1. Strict Single-Page Chunking Constraint

Source File: `backend/services/chunking_service.py` (`chunk_pages`, lines 48–102)

Traditional RAG systems split text by arbitrary token lengths, frequently cutting across physical page boundaries. When an LLM cites that chunk, the system cannot verify which page contained the fact. Scholar strictly enforces:

$$\text{Chunk}_i \subseteq \text{Page}_k \quad (\text{No chunk ever crosses a page boundary})$$

- **Sliding Window Target:** `target_words = 1400`, `overlap_words = 120`.

- **Section Classification** (`_chunk_type`): Scans the chunk text with regex patterns to categorize chunks into `abstract`, `introduction`, `method`, `experiment`, `result`, `limitation`, or `body`.
- **Multi-Document Tracking**: Every chunk stores `page`, `char_start`, `char_end`, `section_title`, and an optional `source_paper_id` to track provenance across cross-referenced papers.

2.2 2. High-Resolution Visual Region Extraction

Source File: `backend/services/pdf_service.py` (`extract_figures`, lines 243–329)

- **Caption Detection**: Uses regex `_CAPTION_RE` to locate `Figure N` and `Table N` blocks.
- **Bounding Box Detection**: `_caption_bbox()` inspects PyMuPDF layout blocks to locate caption coordinates on the page canvas.
- **Clip Region**: Renders from $y_0 - 0.38 \times \text{height}$ (above caption) to $y_1 + 0.12 \times \text{height}$ (below caption).
- **3.0× Zoom Matrix**: Rendered via `fitz.Matrix(3.0, 3.0)` to PNG, ensuring small axis values and mathematical subscripts remain sharp for the Vision LLM.
- **Unified Figure Chunks**: `chunk_figures()` converts figure metadata into indexable chunks with `text = label + ": " + caption` and `is_figure_chunk = True`.

3 The Retrieval Engine & Deep Dive into BM25

3.1 1. Mathematical Formulation of BM25 in Scholar

Source File: `backend/services/retrieval_service.py` (`_bm25_scores`, lines 160–192)

Scholar uses an in-memory, zero-dependency implementation of **Okapi BM25** calibrated for scientific documents:

$$\text{Score}_{\text{BM25}}(D, Q) = \sum_{q_i \in Q} w(q_i) \cdot \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad (1)$$

Where:

- **Inverse Document Frequency (IDF):**

$$\text{IDF}(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

Measures term rarity across all N chunks in the document pool.

- **Term Frequency** $f(q_i, D)$: Count of query token q_i in chunk D .
- **Query Weight** $w(q_i)$: Multiplier for terms repeated in the user prompt (`Counter(query_terms)`).
- **Saturation Parameter** $k_1 = 1.4$: Controls how rapidly term frequency saturates.
- **Length Normalization** $b = 0.72$: Penalizes long documents relative to average document length `avgdl`.

3.2 2. Why BM25 Outperforms Dense Embeddings in Scientific RAG

Scientific literature presents specific challenges that degrade standard dense vector retrieval:

1. **Semantic Drift on Exact Entities:** Searching for *"WMT14"* or *"TruthfulQA"* causes dense embeddings to retrieve generic translation or QA text rather than the specific benchmark table. BM25 guarantees exact keyword hits.
2. **Mathematical Identifiers:** Subscripts and variable names ($d_k, p_\theta, \text{lr} = 10^{-4}$) lose semantic resolution in dense vector space but match directly in lexical indexes.
3. **Zero GPU/Memory Overhead:** BM25 runs in < 40 ms on CPU using Python standard library primitives (`collections.Counter`, `math`, `re`), reserving all RAM and VRAM for the local LLM.

3.3 3. Multi-Signal Re-Ranking Pipeline

Source File: `backend/services/retrieval_service.py` (`retrieve_chunks`, lines 207–316)

After BM25 selects an initial candidate window of $\text{max}(\text{limit} \times 8, 20)$ chunks, Scholar applies a multi-signal heuristic re-ranker:

```
rerank_score = bm25_score
rerank_score += semantic * 0.03           # 128-dim MD5 hashed cosine similarity
rerank_score += expansion_overlap * 0.05  # Overlap with scientific synonyms
rerank_score += exact_overlap * 0.08     # Overlap with literal query terms

if chunk.get("page") in preferred_pages:
    rerank_score += 1.25                  # User asked for "page 4"
if chunk.get("chunk_type") in section_hints:
    rerank_score += 0.08                  # Section match (Method, Results, etc.)
if any(phrase in lowered for phrase in ("we propose", "we introduce", "we present")):
    rerank_score += 0.08                  # Active contribution claim bonus

# Visual-Cue Multiplier:
if chunk.get("is_figure_chunk") and visual_query:
    rerank_score *= 2.5                  # 2.5x boost for figures on visual
    questions
    rerank_score += 1.8 * label_overlap  # Caption token overlap bonus
```

- **CamelCase Boundary Splitting (L69–84):** Regular expressions split fused scientific names (e.g., *FlashAttention* → *flash, attention, flashattention*).

- **Explicit Figure Override (L101–116):** Queries naming specific figures (e.g., "*Figure 2*") extract the exact identifier and pin the figure chunk to Rank 1.

4 RAG Headquarter: Dual-Path Routing & Inference

Source File: backend/main.py (chat endpoint, lines 760–938)

4.1 1. Dual-Path Query Routing

The orchestrator inspects the top-ranked retrieved chunk:

- **Multimodal Vision Path** (vision_service.py: L87--204): If the top chunk is a figure/table, the backend loads the 3× zoom PNG crop, base64-encodes it, builds a visual QA prompt, and passes it to the local VLM (qwen3.5:9b). If the model is offline, it falls back gracefully to caption-only QA.
- **Text Reasoning Path:** If the top chunk is textual, the system proceeds to sentence-level evidence distillation.

4.2 2. Sentence-Level Evidence Distillation

Source File: backend/main.py (_build_evidence_items, lines 175–238)

The backend splits the top 4 chunks into individual sentences, strips boilerplate noise (author affiliations, copyright notices, arXiv headers), and scores each sentence for query overlap and active claim verbs. The top sentences are formatted with ephemeral identifiers:

```
# Context block injected into the LLM prompt:
[E1 | anchor | p. 3 | chunk_003]
"We train our probe battery across all 12 transformer layers using..."

[E2 | anchor | p. 5 | chunk_005]
"Our model achieves a BLEU score of 28.4 on the WMT14 benchmark..."
```

4.3 3. Indirect Evidence-ID Prompting & Normalization

- **Constrained Prompt (L871–905):** The LLM is strictly instructed: *"Cite paper claims inline only with evidence IDs from the Paper evidence list, such as [E1] or [E2]. Never invent page citations. Do not write [p. 1], [p. 2], or any page number yourself."*
- **Backend Citation Normalizer (_normalize_evidence_citations, L267–300):** The application layer intercepts the output, discards any hallucinated IDs (e.g., [E99]), translates valid IDs into sequential reader references ([1], [2]), and attaches the ground-truth metadata:

```
{
  "ref_id": 1,
  "page": 3,
  "chunk_id": "chunk_003",
  "section_title": "Method",
  "quote": "We train our probe battery across all 12 transformer layers using..."
}
```

5 Canvas Highlighting & Mathematical Semantics

5.1 1. PDF Canvas Word-Level Highlighting

Source File: backend/services/pdf_service.py (render_page_png, lines 472–485)

When a user clicks citation chip [1] in the frontend, the UI calls /api/papers/{id}/page/3.png?highlight=<quote>:

1. Runs exact substring search on the PDF page via PyMuPDF's page.search_for().
2. If line-breaks split the sentence, falls back to sliding-window token matching on page.get_text("words").
3. Merges contiguous word rectangles on the same line and draws a semi-transparent blue highlight overlay directly on the rendered PNG canvas.

5.2 2. Mathematical Semantics & KaTeX Rendering

Source Files: `pdf_service.py` (L382–394) and `frontend/components/ChatBox.tsx` (L48–86)

- **Glyph Normalization:** Unicode minus signs (NFKD `–`) and ligatures (*fi*, *fl*) are normalized during ingestion.
- **LaTeX Formatting:** LLM prompts require standard LaTeX formatting (`$...$` and `$$...$$`).
- **KaTeX Typesetting:** The frontend renders math via `katex.renderToString()` with currency collision protection (`looksLikeMath`) to prevent strings like *"\$50 million"* from rendering as equations.